



LINUX基础

什么是linux

Linux 就是一个操作系统，这个操作系统里头含有最主要的 kernel 以及 kernel 提供的工具！他提供了一个完整的操作系统当中最底层的硬件控制与资源管理的完整架构，这个架构是沿袭 Unix 良好的传统来的，所以相当的稳定而功能强大！

Linux为什么那么稳定

Linux 他有个老前辈，那就是 Unix 家族！有这个前辈的提携，让 Linux 这个小老弟很快的就成为一套稳定而优良的操作系统啦！

如何学习linux

从『头』学起

- ⦿ 先理解一下基础的硬件知识，不用一定要全懂(没那么多时间)，但是至少要『听过、有概念』即可；
- ⦿ 先了解一下 Linux 的基础知识，这些包含了『使用者、Group 的概念』、『权限的观念』，『程序的定义』等等；
- ⦿ 必需至少学会一种以上的编辑器，例如最好学会通用版本的 vi ！
- ⦿ 实际操作 Linux 时，最常用的命令，比如查看日志，查找文件等。
- ⦿ 进一步的话，必定要学习的 Shell ，最好 Shell scripts 也能够了解；

Linux的安装

- 1.一般操作系统本身，运维会帮忙安装好；
- 2.自己安装的话，可以使用VMWare或者VirtualBox等安装虚拟机
- 3.安装最常用的发布版本：银行较为常用的是RedHat Enterprise版本，另外还有CentOS和Ubuntu可选。
- 4.给大家准备了一个安装好的VMWare虚拟机可以直接使用。（rhel6.8）

Linux的开机流程

- ◉ 加载 BIOS 的硬件信息；
- ◉ 读取 MBR 的 Kernel Loader (亦即是 lilo, grub, spfdisk 等等)开机信息；
- ◉ 加载 Kernel 的操作系统核心信息；
- ◉ Kernel 执行 init 程序并取得 run-level 信息；
- ◉ init 执行 /etc/rc.d/rc.sysinit 文件；
- ◉ 启动核心的外挂式模块 (/etc/modules.conf)；
- ◉ init 执行 run-level 的各个批次档(Scripts)；
- ◉ **init 执行 /etc/rc.d/rc.local 文件；**
- ◉ 执行 /bin/login 程序；
- ◉ 登入之后开始以 Shell 控管主机。



LINUX 各个目录的作用及内容

根目录 “/”

这就是根目录。对你的电脑来说，有且只有一个根目录。所有的东西，我是说所有的东西都是从这里开始。举个例子：当你在终端里输入“/home”，你其实是在告诉电脑，先从/（根目录）开始，再进入到home目录。

/bin 二进制可执行命令

/ bin目录,(bin是binary -二进制的简称),包含了引导启动所需的命令或普通用户可能用的命令(可能在引导启动后).这些命令都是二进制文件的可执行程序,多是系统中重要的系统文件.

/boot目录

/boot目录存放引导加载器(bootstrap loader)使用的文件,如lilo,核心映像也经常放在这里,而不是放在根目录中.但是如果有许多核心映像,这个目录就可能变得很大.

/dev设备特殊文件(文件系统)

/dev目录包括所有设备的设备文件,即设备驱动程序,用户通过这些文件访问外部设备.设备文件用特定的约定命名,这在设备列表中有说明.设备文件在安装时由系统产生,以后可以用/dev/makedev 描述

.

/etc 系统管理和配置文件

/etc目录存放着各种系统配置文件,其中包括了用户信息文件/etc/passwd,系统初始化文件/etc/rc等.linux正是有了这些文件才得以正常运行.

/home 用户主目录的基点

这是用户主目录的基点,例如: user的主目录就是/home/user,可以用~user表示.

/lib 标准程序设计库（动态链接共享库）

- /lib目录是根文件系统上的程序所需的共享库,存放了根文件系统程序运行所需的共享文件. 这些文件包含了可被许多程序共享的代码,以避免每个程序都包含有相同的子程序的副本,故可以使得可执行文件变得更小,节省空间
- /lib/modules 目录包含系统核心,可加载各种模块,尤其是那些在恢复损坏的系统时重新引导系统所需的模块(例如网络 and 文件系统驱动).

/media

有些linux的发行版使用这个目录来挂载那些usb接口的移动硬盘（包括U盘）、CD/DVD驱动器等等。

/mnt 系统提供这个目录是让用户临时挂载其他的文件系统

/mnt目录是系统管理员临时安装(mount)文件系统的安装点.程序并不自动支持安装到/mnt ./mnt 下面可以分为许多子目录,例如/mnt/exta 可能是使用ext2文件系统的软驱,/mnt/cdrom 光驱等.

/opt

这里主要存放那些可选的程序。你想尝试最新的firefox测试版吗？那就装到/opt目录下吧，这样，当你尝试完，想删掉firefox的时候，你就可以直接删除它，而不影响系统其他任何设置。安装到/opt目录下的程序，它所有的数据、库文件等等都是放在同个目录下面。

/proc 虚拟的目录

是系统内存的映射.可直接访问这个目录来获取系统信息. 可以在该目录下获取系统信息，这些信息是在内存中由系统自己产生的，该目录的内容不在硬盘上而在内存里。

/root 系统管理员的主目录

它是超级用户的目录.

/sbin 系统管理命令

这里存放的是系统管理员使用的管理程序。

/sbin目录类似/bin,也用于存储二进制文件。其中的大部分文件多是系统管理员使用的基本的系统程序,普通用户想要使用时,也要得到允许才行,但一般不给普通用户使用。

/tmp 公用的临时文件存储点

/tmp 目录存放程序在运行时产生的信息和数据.但在引导启动后,运行的程序最好使用/var /tmp来代替/tmp,前者可能拥有一个更大的磁盘空间.

/usr 存放应用程序和文件

所有程序安装在/usr 目录里./usr 里的所有文件一般来自linux发行版(distribution); 本地安装的程序和其他东西在/usr/local 下, 这样可以在升级新版系统或新发行版时无须重新安装全部程序./usr 目录下的许多内容都是可选的.

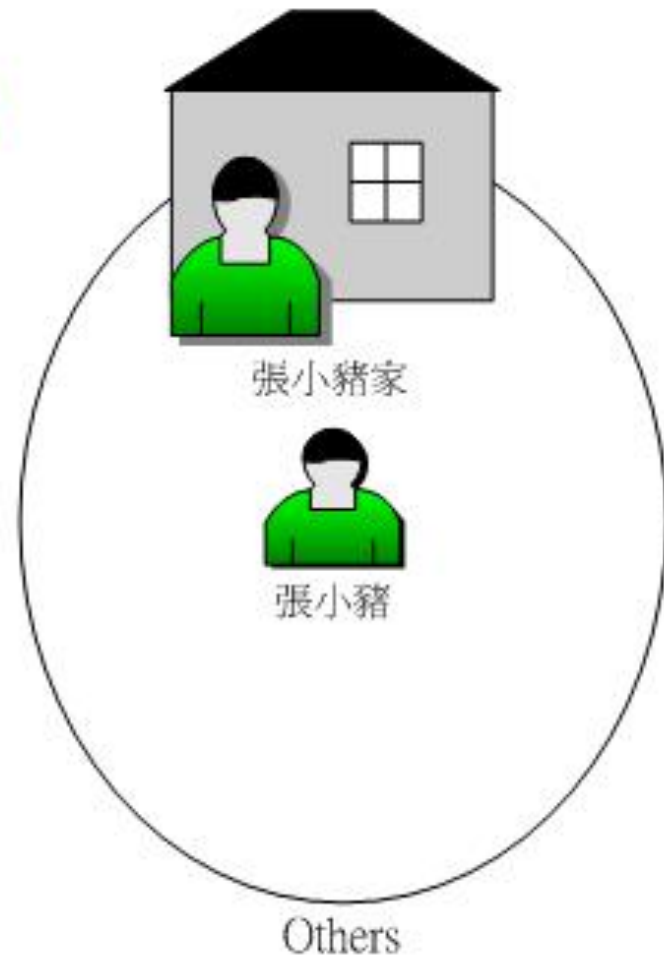
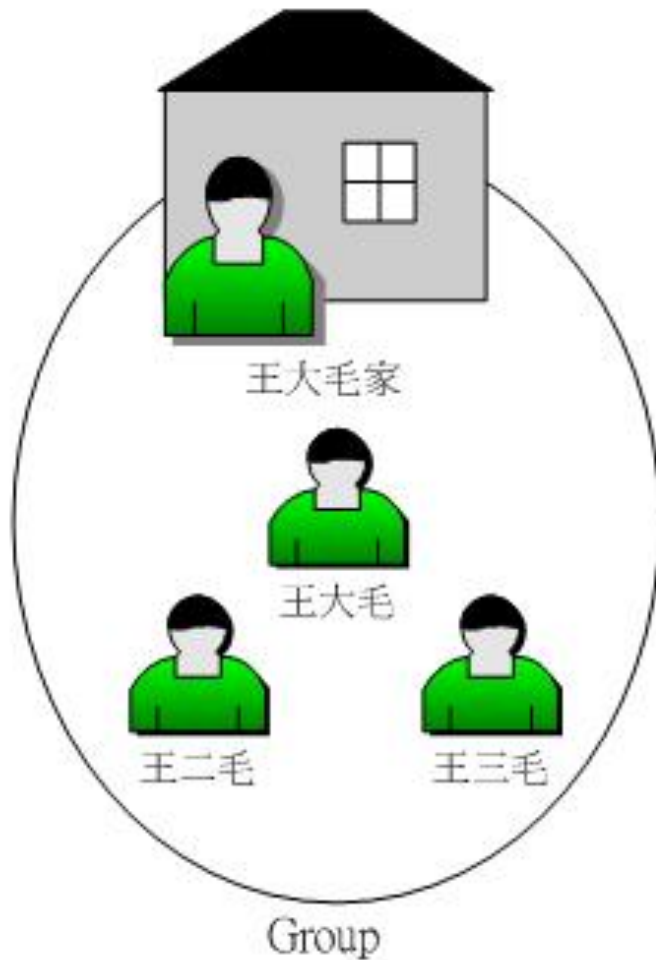
/var 某些大文件的溢出区

/var 包含系统一般运行时要改变的数据.通常这些数据所在的目录的大小是要经常变化 或扩充的.原来 /var目录中有些内容是在/usr中的,但为了保持/usr目录的相对稳定,就把那些需要经常改变的目录放到 /var中了.每个系统是特定的,即不通过网络与其他计算机共享



LINUX的文件属性及目录配置

User & Group



Linux 文件属性

```
shum@sol:~$ ls -l
total 20
drwx----- 2 shum staff 4096 Jan 16 22:04 Mail
drwx----- 3 shum staff 4096 Jan 16 14:15 csc128
drwxr-xr-x  2 shum staff 4096 Jan 13 16:42 public
drwxr-xr-x  2 shum staff 4096 Jan 16 14:07 public_html
-rw-r--r--  1 shum staff 628 Jan 15 20:04 verse
```

The diagram illustrates the components of the `ls -l` command output:

- file type**: Indicated by the first character of the permissions string (e.g., `d` for directory, `-` for regular file).
- number of hard links**: Indicated by the first number in the permissions string (e.g., `2`).
- user (owner) name**: Indicated by the user name (e.g., `shum`).
- group name**: Indicated by the group name (e.g., `staff`).
- size**: Indicated by the file size in bytes (e.g., `4096`).
- date/time last modified**: Indicated by the date and time (e.g., `Jan 16 22:04`).
- filename**: Indicated by the file name (e.g., `Mail`).
- permissions**: Indicated by the permissions string (e.g., `drwxr-xr-x`).
- other (everyone) permissions**: Indicated by the last three characters of the permissions string (e.g., `r-x`).
- group permissions**: Indicated by the middle three characters of the permissions string (e.g., `wxr`).
- user permissions**: Indicated by the first three characters of the permissions string (e.g., `drwx`).
- executable**: Indicated by the `x` character in the permissions string.
- writable**: Indicated by the `w` character in the permissions string.
- readable**: Indicated by the `r` character in the permissions string.

强调第一档

文件 类型	属主 权限			属组 权限			其他用户 权限		
0	1	2	3	4	5	6	7	8	9
d	rwX			r-X			r-X		
目录 文件	读	写	执行	读	写	执行	读	写	执行

例子

- 例题一：如果有下面的两个文件：

```
-rw-r--r--  1 root    root      238 Jun 18 17:22 test.txt
```

```
-rwxr-xr--  1 test1   testgroup 5238 Jun 19 10:25 ping_tsai
```

请说明两个文件的拥有者与其相关的权限为何？

答：

- 文件『 test.txt 』的拥有者为 root ， Group为 root 。至于权限方面则只有 root 这个账号可以存取此文件，其它人则仅能读此文件；
- 另一个文件『 ping_tsai 』的拥有者为 test1 ， 而Group为 testgroup。其中， test1 可以针对此文件具有可读可写可执行的权力，而同Group的 test2, test3 两个人与 test1 同样是 testgroup 的Group账号，则仅可读可执行但不能写（亦即不能修改），至于非 testgoup 这一个Group的人则仅可以读，不能写也不能执行！

如何改变文件权限

chgrp : 改变文件所属Group

chown : 改变文件所属人

chmod : 改变文件的属性、 SUID 、等等的特性

改变所属Group, chgrp

Linux chgrp（英文全拼：change group）命令用于变更文件或目录的所属群组。

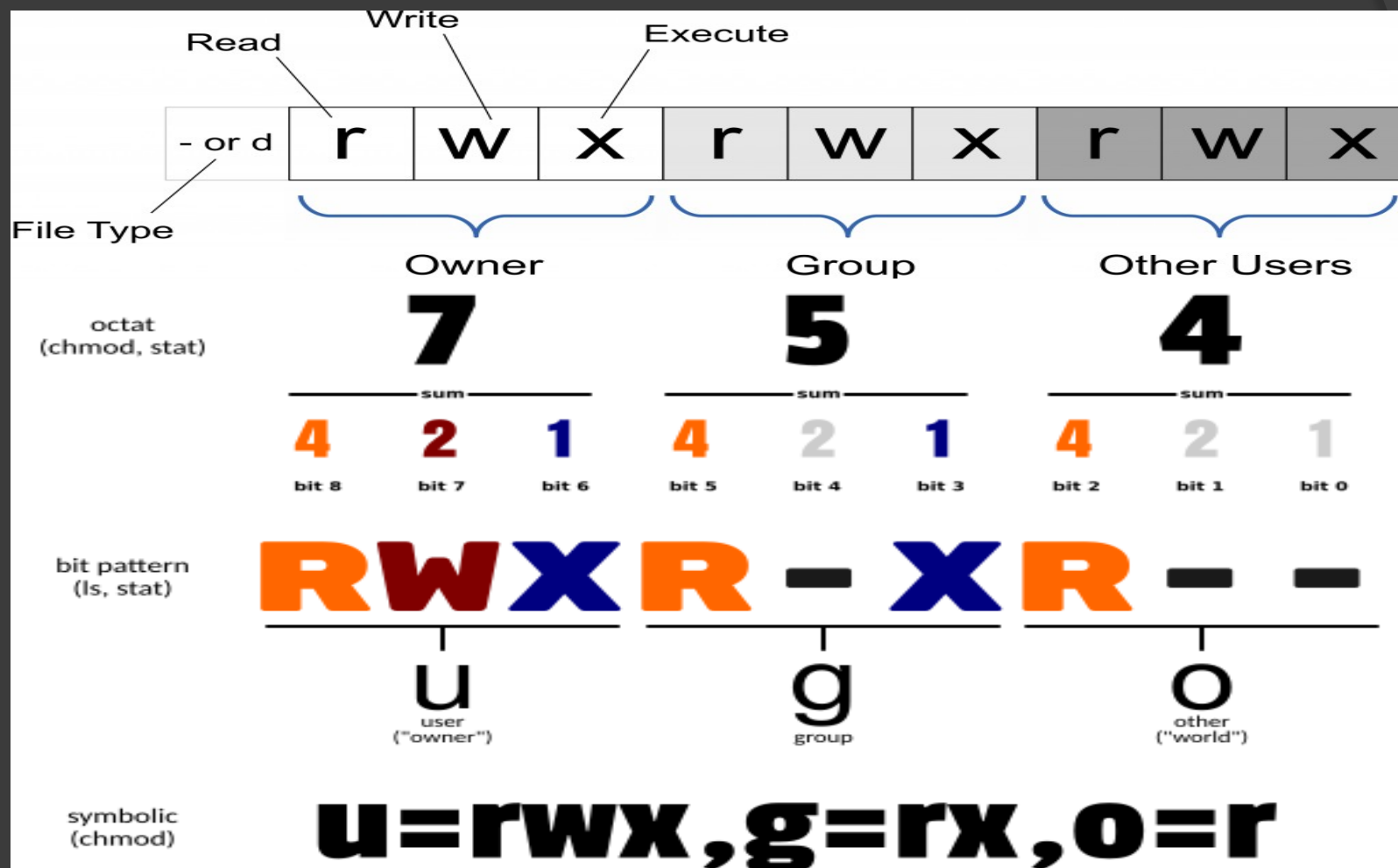
与 [chown](#) 命令不同，chgrp 允许普通用户改变文件所属的组，只要该用户是该组的一员。

```
[root@localhost temp]# ls -l
总用量 4
-rw-r--r--. 1 root root 4 8月 23 17:09 app.log
[root@localhost temp]# chgrp -v bin app.log
changed group of "app.log" from root to bin
[root@localhost temp]# ls -l
总用量 4
-rw-r--r--. 1 root bin 4 8月 23 17:09 app.log
```

改变文件所有者, chown

- 把 /var/run/httpd.pid 的所有者设置 root:
`chown root /var/run/httpd.pid`
- 将文件 file1.txt 的拥有者设为 admin, Group为 root :
`chown admin:root file1.txt`
- 将当前前目录下的所有文件与子目录的拥有者皆设为 admin , 群体的使用者 root:
`chown -R admin:root *`

改变九个属性 chmod



改变九个属性 chmod 举例

- 将文件 file1.txt 设为所有人皆可读取：
`chmod a+r file1.txt`
- 将文件 file1.txt 与 file2.txt 设为该文件拥有者，与其所属同一个Group可写入，但其他以外的人则不可写入：
`chmod ug+w,o-w file1.txt file2.txt`
- 将目前目录下的所有文件与子目录皆设为任何人可读取：
`chmod -R a+r *`
- 此外chmod也可以用数字来表示权限如：
`chmod a=rwx file`和`chmod 777 file`效果相同
`chmod ug=rwx,o=x file`和`chmod 771 file`效果相同



LINUX的文件与目录管理

绝对路径与相对路径

绝对路径：路径的写法『一定由根目录 / 写起』

- 例如： /usr/share/doc 这个目录。

相对路径：路径的写法『不是由 / 写起』

- 例如由 /usr/share/doc 要到 /usr/share/man 底下时，
可以写成：『cd ../man』这就是相对路径的写法

几个常用命令

- **cd** 变换目录
- **pwd** 显示目前的目录
- **mkdir** 建立一个新目录
- **rmdir** 删除一个里面是空的空目录——层层删除
- **rm---**将所有目录下的东西都删除掉

cd

Linux cd（英文全拼：change directory）命令用于切换当前工作目录。

其中 dirName 表示法可为绝对路径或相对路径。若目录名称省略，则变换至使用者的 home 目录 (也就是刚 login 时所在的目录)。

另外，~ 也表示为 home 目录 的意思，. 则是表示目前所在的目录，.. 则表示目前目录位置的上一层目录。

语法

```
cd [dirName]
```

dirName：要切换的目标目录。

实例

跳到 /usr/bin/：cd /usr/bin

跳到自己的 home 目录：cd ~

跳到目前目录的上上两层：cd ../../

pwd

Linux pwd（英文全拼：print work directory）命令用于显示工作目录。
执行 pwd 指令可立刻得知您目前所在的工作目录的绝对路径名称。

⚠注意：

执行变更类的命令之前，一定要使用pwd命令确认当前所在的目录

```
[admin@localhost ~]$ cd /home  
[admin@localhost home]$  
[admin@localhost home]$ pwd  
/home
```

mkdir

Linux mkdir（英文全拼：make directory）命令用于创建目录。

语法

```
mkdir [-p] dirName
```

参数说明：

-p 确保目录名称存在，不存在的就建一个。

在工作目录下，建立一个名为 work 的子目录：

```
mkdir work
```

在工作目录下的 work2 目录中，建立一个名为 test 的子目录。

若 work2 目录原本不存在，则建立一个。

（注：本例若不加 -p 参数，且work2 目录不存在，则产生错误。）

```
mkdir -p work2/test
```

rmmdir

```
[root @test /root ]# rmmdir [-p] [目录名称]
```

参数说明:

-p : 将上层的目录也删除吧!

范例:

```
[root @test /root]# rmmdir test<==删除名称为 test 的目录
```

```
[root @test tmp]# ll
```

```
drwxrwxr-x    3 test    test          4096 Feb  6 20:48 test1/
```

```
[root @test tmp]# rmmdir test1
```

```
rmmdir: `test1': Directory not empty
```

```
[root @test tmp]# rmmdir -p test1/test2/test3/test4
```

```
[root @test tmp]$ ll
```

文件与目录管理

常用命令：

ls 显示文件名称、属性等

cp 拷贝文件或目录

rm 删除文件或目录

mv 移动文件或目录

Ls

Linux ls（英文全拼：list files）命令用于显示指定工作目录下之内容（列出目前工作目录所含之文件及子目录）。

语法

ls [-alrtAFR] [name...] 参数：

- a 显示所有文件及目录（. 开头的隐藏文件也会列出）
- l 除文件名称外，亦将文件型态、权限、拥有者、文件大小等资讯详细列出
- r 将文件以相反次序显示(原定依英文字母次序)
- t 将文件依建立时间之先后次序列出
- A 同 -a，但不列出 "." (目前目录) 及 ".." (父目录)
- F 在列出的文件名称后加一符号；例如可执行档则加 "*", 目录则加 "/"
- R 若目录下有文件，则以下之文件亦皆依序列出

```
[root@localhost test]# ls -ltr
```

总用量 12

```
-rw-r--r--. 1 root root 93 5月 18 17:07 test.send.bak
-rw-r--r--. 1 root root 93 5月 18 17:07 test.send
-rw-r--r--. 1 root root 119 5月 18 17:08 replace.sh
```

rm

Linux rm（英文全拼：remove）命令用于删除一个文件或者目录。

rm [options] name...参数：

-f 即使原档案属性设为唯读，亦直接删除，无需逐一确认。

-r 将目录及以下之档案亦逐一删除。

注意：传说中的删库命令 rm -rf 一定要慎用！！

```
[root@localhost work]# ls -l
总用量 0
-rw-r--r--. 1 root root 0 8月 26 15:24 app.txt
drwxr-xr-x. 2 root root 80 8月 26 15:19 test
drwxr-xr-x. 2 root root 80 8月 26 15:20 test2
[root@localhost work]# rm -f app.txt
[root@localhost work]# rmdir test2
rmdir: 删除 "test2" 失败：目录非空
[root@localhost work]# rm -rf test2
[root@localhost work]# mkdir test3
[root@localhost work]# rmdir test3
[root@localhost work]# ls -l
总用量 0
drwxr-xr-x. 2 root root 80 8月 26 15:19 test
```

cp

Linux cp（英文全拼：copy file）命令主要用于复制文件或目录。

cp [options] source dest 参数说明：

-a：此选项通常在复制目录时使用，它保留链接、文件属性，并复制目录下的所有内容。其作用等于dpR参数组合。

-d：复制时保留链接。

-f：覆盖已经存在的目标文件而不给出提示。

-p：除复制文件的内容外，还把修改时间和访问权限也复制到新文件中。

-r：若源文件是一个目录文件,此时将复制该目录下所有的子目录和文件。

-l：不复制文件，只是生成链接文件。

注意：用户使用该指令复制目录时，必须使用参数 **-r** 或者 **-R**。

```
[root@localhost work]# ls -l
总用量 0
drwxr-xr-x. 2 root root 80 8月 26 15:19 test
[root@localhost work]# cp -r test test2
[root@localhost work]# ls -l
总用量 0
drwxr-xr-x. 2 root root 80 8月 26 15:19 test
drwxr-xr-x. 2 root root 80 8月 26 15:20 test2
```


mv

Linux mv（英文全拼：move file）命令用来为文件或目录改名、或将文件或目录移入其它位置。

mv [options] source dest 参数说明：

- b: 当目标文件或目录存在时，在执行覆盖前，会为其创建一个备份。
- f: 如果指定移动的源目录或文件与目标的目录或文件同名，不会询问，直接覆盖旧文件。
- n: 不要覆盖任何已存在的文件或目录。
- u: 当源文件比目标文件新或者目标文件不存在时，才执行移动操作。

```
[root@localhost work]# ls -l
总用量 0
drwxr-xr-x. 2 root root 80 8月 26 15:19 test
drwxr-xr-x. 2 root root 80 8月 26 15:28 test2
[root@localhost work]# mv test2 test3
[root@localhost work]# ls -l
总用量 0
drwxr-xr-x. 2 root root 80 8月 26 15:19 test
drwxr-xr-x. 2 root root 80 8月 26 15:28 test3
```

观看文件内容

cat 由第一行开始显示文件内容

tac 从最后一行开始显示

more 一页一页的显示文件内容

less 与 more 类似，但是可以往前翻页！

head 只看头几行

tail 只看尾巴几行

nl 显示的时候，顺道输出行号！

od 以二进制的方式读取文件内容！

cat

cat（英文全拼：concatenate）命令用于连接文件并打印到标准输出设备上。

cat [-AbeEnstTuv] [--help] [--version] fileName 参数说明：

-n：由 1 开始对所有输出的行数编号。

-A：显示所有的不能打印的符号，将 TAB 字符显示为 ^I，在每行结束处显示 \$，windows换行显示为 M- 符号

```
[root@localhost test]# cat test.send.bak
"rptTp":"A008","rptTp":"A007"
"lvl2Lb1":"B002";"A0081","BA008"
"rptTp":"A008","rptTp":"A007"
```

```
[root@localhost test]# cat -A test.send.bak
"rptTp":"A008","rptTp":"A007"$
"lvl2Lb1":"B002";"A0081","BA008"$
"rptTp":"A008","rptTp":"A007"$
```

more

Linux more 命令类似 cat，以一页一页的形式显示，而最基本的指令就是按空白键（space）就往下一页显示，按 b 键就会往回（back）一页显示，而且还有搜寻字串的功能（与 vi 相似）。

more [-dlfpcsu] [-num] [+/pattern] [+linenum] [fileNames..]参数：

-num 一次显示的行数

+/pattern 在每个文档显示前搜寻该字串（pattern），然后从该字串之后开始显示

+num 从第 num 行开始显示

```
[root@localhost logs]# more -10 catalina.out
Java HotSpot(TM) 64-Bit Server VM warning: ignoring option PermSize=512m; support was removed in 8.0
Java HotSpot(TM) 64-Bit Server VM warning: ignoring option MaxPermSize=2048m; support was removed in 8.0
27-Jul-2021 20:05:53.337 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Server version:      Apache Tomcat/8.0.48
27-Jul-2021 20:05:53.345 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Server built:      Nov 30 2017 16:26:50 UTC
27-Jul-2021 20:05:53.345 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Server number:    8.0.48.0
27-Jul-2021 20:05:53.345 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log OS Name:          Linux
27-Jul-2021 20:05:53.346 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log OS Version:      3.10.0-1062.el7.x86_64
27-Jul-2021 20:05:53.346 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Architecture:   amd64
27-Jul-2021 20:05:53.346 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Java Home:       /usr/java/jdk1.8.0_11/jre
27-Jul-2021 20:05:53.347 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log JVM Version:     1.8.0_11-b12
--More--(0%)
```

less

less 与 more 类似，less 可以随意浏览文件，支持翻页和搜索，支持向上翻页和向下翻页。

语法

less [参数] 文件

head

head 命令可用于查看文件的开头部分的内容，有一个常用的参数 **-n** 用于显示行数，默认为 10，即显示 10 行的内容。

head [参数] [文件] 参数：

-n<行数> 显示的行数。

要显示 app.log 文件的开头 10 行，请输入以下命令：

```
head app.log
```

显示 app.log 文件的开头 50 行，请输入以下命令：

```
head -n 50 app.log
```

tail

tail 命令可用于查看文件的结尾部分的内容，有一个常用的参数 **-n** 用于显示行数，默认为 10，即显示 10 行的内容。

tail [参数] [文件] **参数：**

-n<行数> 显示的行数。

要显示 app.log 文件的结尾 10 行，请输入以下命令：

```
tail app.log
```

显示 app.log 文件的结尾50 行，请输入以下命令：

```
tail -n 50 app.log
```

监视文件

```
tail -f app.log
```

查找目录，文件或内容

- which** 查看可执行文件的位置
- find** 实际搜寻硬盘去查询文件名称
- grep** 搜寻文件内容

which

Linux which命令用于查找文件。

which指令会在环境变量\$PATH设置的目录里查找符合条件的文件。

语法

which [文件...]

```
[root@localhost logs]# which java
/usr/bin/java
```

find

Linux find 命令用来在指定目录下查找文件。任何位于参数之前的字符串都将被视为欲查找的目录名。如果使用该命令时，不设置任何参数，则 find 命令将在当前目录下查找子目录与文件。并且将查找到的子目录和文件全部进行显示。

```
find path -option [ -print ] [ -exec -ok command ] {} \;
```

参数说明：

- ctime n : 在过去n天内被修改过的文件
- name name, -iname name : 文件名称符合 name 的文件。iname 忽略大小写
- size n : 文件大小 是 n 单位，b 代表 512 位元组的区块，c 表示字元数，k 表示 kilo bytes，w 是二个位元组。
- type c : 文件类型是 c 的文件。
 - d: 目录
 - f: 一般文件
 - l: 符号连结

find

实例

将当前目录及其子目录下所有文件后缀为 .java 的文件列出来:

```
# find . -name "*.java"
```

将当前目录及其子目录中的所有文件列出:

```
# find . -type f
```

将当前目录及其子目录中的所有目录列出:

```
# find . -type d
```

将当前目录及其子目录下所有最近 20 天内更新过的文件列出:

```
# find . -ctime -20
```

grep

Linux grep 命令用于查找文件里符合条件的字符串。

grep 指令用于查找内容包含指定的范本样式的文件，如果发现某文件的内容符合所指定的范本样式，预设 grep 指令会把含有范本样式的那一行显示出来。

语法

```
grep [-abcEFHhIlLnqrsVwxy][[-A<显示行数>][[-B<显示列数>][[-C<显示列数>][[-d<进行动作>][[-e<范本样式>][[-f<范本文件>][--help][范本样式][文件或目录...]
```

参数：

- l 或 --file-with-matches：列出文件内容符合指定的样式的文件名称。
- L 或 --files-without-match：列出文件内容不符合指定的样式的文件名称。
- n 或 --line-number：在显示符合样式的那一行之前，标示出该行的列数编号。
- r 或 --recursive：此参数的效果和指定"-d recurse"参数相同。
- v 或 --invert-match：显示不包含匹配文本的所有行。

该命令支持正则表达式

grep

正向和反向查找文本

```
[root@localhost test]# cat test.send.bak
"rptTp":"A008","rptTp":"A007"
"lv12Lb1":"B002";"A0081","BA008"
"rptTp":"A008","rptTp":"A007"
[root@localhost test]# grep -v BA008 test.send.bak
"rptTp":"A008","rptTp":"A007"
"rptTp":"A008","rptTp":"A007"
[root@localhost test]# grep BA008 test.send.bak
"lv12Lb1":"B002";"A0081","BA008"
```

查找包含对应文本的文件名

```
[root@localhost logs]# grep -l bondDuraRepayPlanCheck ./ * | head
./catalina.out
./dump.bin
./localhost.2021-08-05.log
./localhost.2021-08-12.log
./localhost_access_log.2021-07-28.txt
./localhost_access_log.2021-07-29.txt
./localhost_access_log.2021-07-30.txt
./localhost_access_log.2021-08-02.txt
./localhost_access_log.2021-08-03.txt
./localhost_access_log.2021-08-04.txt
```



LINUX系统状态管理

SU

Linux su（英文全拼：switch user）命令用于变更为其他使用者的身份，除 root 外，需要键入该使用者的密码。

`su [-fmp] [-c command] [-s shell] [--help] [--version] [-] [USER [ARG]]`

参数说明：

`-m -p` 或 `--preserve-environment` 执行 su 时不改变环境变数

`-s shell` 或 `--shell=shell` 指定要执行的 shell（bash csh tcsh 等），预设值为 `/etc/passwd` 内的该使用者（USER）shell

USER 欲变更的使用者帐号,内定是 root

ARG 传入新的 shell 参数

示例

切换为root:

su

切换为用户1

su - user1

ps

Linux ps（英文全拼：process status）命令用于显示当前进程的状态，类似于 windows 的任务管理器。

ps [options] [--help]

参数：

ps 的参数非常多，在此仅列出几个常用的参数并大略介绍含义

-A 列出所有的进程

-w 显示加宽可以显示较多的资讯

-au 显示较详细的资讯

-aux 显示所有包含其他使用者的行程 ps -ef

```
[root@localhost test]# ps -aux | head -n 4
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	ST
root	1	0.0	0.0	195536	4928	?	Ss
root	2	0.0	0.0	0	0	?	S
root	4	0.0	0.0	0	0	?	S<

kill

Linux kill 命令用于删除执行中的程序或工作。

kill 可将指定的信息送至程序。预设的信息为 SIGTERM(15)，可将指定程序终止。若仍无法终止该程序，可使用 SIGKILL(9) 信息尝试强制删除程序。

kill [-s <信息名称或编号>][程序]

参数说明：

-s <信息名称或编号> 指定要送出的信息。

[程序] [程序]可以是程序的PID或是PGID，也可以是工作编号。

最常用的信号是：

实例

杀死进程

kill 12345

强制杀死进程

kill -KILL 12345 或者 kill -9 12345

free

Linux free命令用于显示内存状态。

free指令会显示内存的使用情况，包括实体内存，虚拟的交换文件内存，共享内存区段，以及系统核心使用的缓冲区等。

语法

free [-bkmotV][*-s* <间隔秒数>]

参数说明：

- b 以Byte为单位显示内存使用情况。
- k 以KB为单位显示内存使用情况。
- m 以MB为单位显示内存使用情况。
- h 以合适的单位显示内存使用情况，最大为三位数，自动计算对应的单位值。

```
[root@localhost test]# free
```

	total	used	free	shared	buff
Mem:	65845908	5397844	5538412	126036	54
Swap:	23392252	3434156	19958096		

top

Linux top命令用于实时显示 process 的动态。

top [-] [d delay] [q] [c] [S] [s] [i] [n] [b]

参数说明：

d：改变显示的更新速度，或是在交谈式指令列(interactive command)按 s

q：没有任何延迟的显示速度，如果使用者是有 superuser 的权限，则 top 将会以最高的优先序执行

c：切换显示模式，共有两种模式，一是只显示执行档的名称，另一种是显示完整的路径与名称

S：累积模式，会将已完成或消失的子行程 (dead child process) 的 CPU time 累积起来

s：安全模式，将交谈式指令取消, 避免潜在的危机

i：不显示任何闲置 (idle) 或无用 (zombie) 的行程

n：更新的次数，完成后将会退出 top

b：批次档模式，搭配 "n" 参数一起使用，可以用来将 top 的结果输出到档案内



LINUX压缩命令的运用

Linux 压缩文件

- ◎ *.tar tar 程序打包的数据，并没有压缩过；
- ◎ *.tar.gz tar 程序打包的文件，其中并且经过 gzip 的压缩

创建压缩文件

```
tar czvf testuser.tar.gz /home/testuser/*
```

解压

```
tar xzvf testuser.tar.gz
```

查看

```
tar tvf testuser.tar.gz
```

tar语法

Linux tar（英文全拼：tape archive）命令用于备份文件。

tar 是用来建立，还原备份文件的工具程序，它可以加入，解开备份文件内的文件。

-c或--create 建立新的备份文件。

-f<备份文件>或--file=<备份文件> 指定备份文件。

-t或--list 列出备份文件的内容。

-v或--verbose 显示指令执行过程。

-x或--extract或--get 从备份文件中还原文件。

-z或--gzip或--ungzip 通过gzip指令处理备份文件。

--delete 从备份文件中删除指定的文件。

--exclude=<范本样式> 排除符合范本样式的文件。

--remove-files 文件加入备份文件后，就将其删除。

tar 范例 1

```
[root @test /root]# tar -cvf directory.tar directory
```

只将目录整合打包成一个档案

```
[root @test /root]# tar -zcvf directory.tar.gz directory
```

除了将目录打包外，同时以 gzip 压缩

```
[root @test /root]# tar -zcvf filename.tar.gz /home/test/*
```

将 /home/test/ 这个目录下的档案全部打包并压缩成为一个 filename.tar.gz 的档案

```
[root @test /root]# tar -xvf directory.tar
```

解 tar 的封包，请注意，由于没有 gzip (.tar 而非 .tar.gz) 的作用，所以只要使用 -xvf 即可！不需要加上 z，否则会显示有问题！

```
[root @test /root]# tar -zxvf directory.tar.gz
```

这个就是有加上 gzip 的压缩的结果！所以需要加上 -z 哟！

```
[root @test /root]# tar -ztvf directory.tar.gz
```

这个 t 可以用来查看 tar 里面的档案信息呢！而不需要将他解开！

```
[root @test /root]# tar -zcvPf home.tar.gz /home
```

则建立起来的压缩档内档案为绝对路径

请注意，使用这个 P 的参数时，不要将 P 加在 f 后面，因为

f 之后要立即接档名才行喔！

tar范例2

```
[root @test /root]# tar -N '2002/06/25' -zcvf home.tar.gz /home
```

上面是说 在 /home 这个目录中，比 2002/06/25 日还要新的档案才会被打包进入 home.tar.gz 这个档案中！

```
[root @test /root]# tar -zcvf host.tar.gz / --exclude /mnt --exclude /proc
```

上面是说，将根目录的所有数据都打包进 host.tar.gz 这个档案中，但是 /mnt 及 /proc 则不打包！

```
[root @test /root]# tar -cvf - /home | tar -xvf -
```

上面的意思是『将 /home 打包之后，直接解压缩在 /root 底下！』嘿嘿！不需要再建立一次中间档案！不过，使用上面的语法最好使用『绝对路径』，比较不会有问题！这个方式适合不想要建立中间档案时！

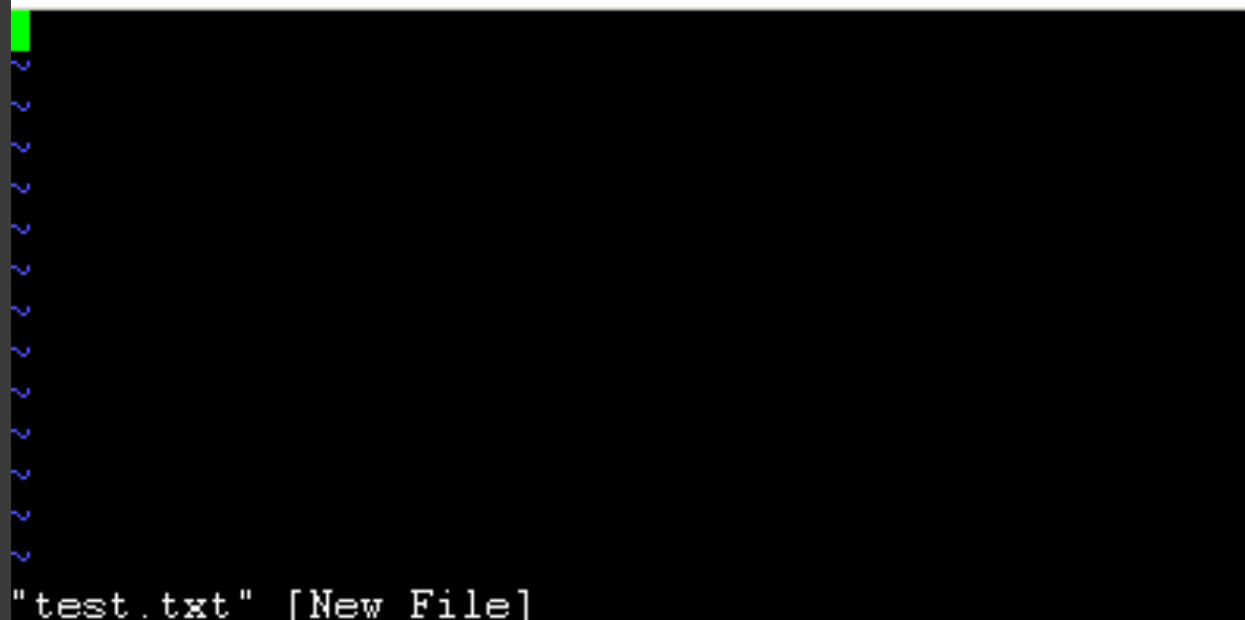


LINUX VI的使用

使用 vi 进入一般模式

```
[root @test root]# vi test.txt
```

直接输入『vi 档名』即可进入 vi 了！如下图所示，左下角还会显示这个档案目前的状态！如果是新建档案会显示 [New File]，如果是已存在的档案，则会显示目前的文件名、行数与字符数，例如：『"/etc/man.config" 145L, 4614C』



按下 i 进入编辑模式，开始编辑文字

在一般模式之中，只要按下 i, o, a 等字符，就可以进入编辑模式了！在编辑模式当中，您可以发现在左下角会出现 - INSERT - 的画面，那就是可以输入任意字符的提示啰！这个时候，键盘上除了 [Esc] 这个按键之外，其它的按键都可以视作为一般的输入按钮了，所以您可以进行任何的编辑啰！（注：在 vi 里面，[tab] 这个按钮所得到的结果与空格符所得到的结果是不一样的，特别强调一下！）

[illegible]

按下 [ESC] 按钮回到一般模式

好了，假设我已经按照上面的样式给他编辑完毕了，那么应该要如何退出呢？是的！没错！就是给他按下 [Esc] 这个按钮即可！马上你就会发现画面左下角的 – INSERT – 不见了！

在一般模式中按下 :wq 储存后
离开 vi

OK，我们要存档了，存盘并离开的指令很简单，输入『:wq』即可存档离开！（注意了，按下：该光标就会移动到最底下一行去！）这时你在提示字符后面输入『ls -l』即可看到我们刚刚建立的 test.txt 档案啦！整个图示有点像底下这样：

```
I am VBird  
I am very happy to write this class for you!  
If you have nay question, please contact with me.  
VBird 2002/10/24
```

:wq



LINUX 命令进阶

输出重定向

定义：将命令的标准输出结果保存到指定的文件中，而不是直接显示在显示器上。
输出重定向使用>和>>操作符。

语法：cmd > filename，表示将标准输出写入文件，如果此文件已存在，将覆盖原文件的内容。如果不存在，则新建该文件。

cmd >> filename，表示将标准输出追加到文件，如果文件不存在，则新建该文件。

实例：查看当前主机的CPU类型保存到cpu.txt文件中，而不是直接显示到屏幕上。

//如果想要直接查看CPU信息

```
cat /proc/cpuinfo
```

//那么相应的输出重定向如下

```
cat /proc/cpuinfo > cpu.txt
```


命令组合

查询xxx进程

`ps -ef | grep xxx`

比如查询正在运行的Java进程

```
[root@localhost logs]# ps -ef | grep java | grep -v grep
root      2762   2761   2 09:11 ?                00:13:37 /usr/java/jdk1.8.0_11/jre/b
logging.manager=org.apache.juli.ClassLoaderLogManager -Xms3g -Xmx3g -XX:Per
.catalina.webresources -Dorg.apache.catalina.security.SecurityListener.UMAS
cat-9.0.16/bin/tomcat-juli.jar -Dcatalina.base=/home/apache-tomcat-9.0.16 -l
atalina.startup.Bootstrap start
root      23118      1 15 14:43 pts/1      00:24:01 /usr/java/jdk1.8.0_11/jre/b
logging.manager=org.apache.juli.ClassLoaderLogManager -Xms3g -Xmx3g -XX:Per
```

查看log里面的Exception信息并逐页查看

```
[root@localhost logs]# grep Exception catalina.out | more 20
INFO 20:08:00.924 | Mapped "{[/bond/bondDuraRepayPlanCheck/saveOrUpdate]}"
.controller.BondDuraRepayPlanCheckController.saveOrUpdateProd(com.joyintech
ch.activiti.service.entity.SubmitProcessInput) throws java.text.ParseExcept
appingRegistry.register(AbstractHandlerMethodMapping.java:544)
```

命令组合

tail和grep结合，监控日志中有关键字出现的行

```
[root@localhost logs]# tail -f catalina.out | grep INFO
INFO 17:21:13.339 | 请求路径=/archiveFile/upload/save 来源IP=1
61e0ce85fc4fe0aaf5bdba4f9c1653, sceneType=0, file_id=0, subType
0-exec-7 | com.joyintech.platform.interceptor.BaseInterceptor.a
INFO 17:21:13.340 | 本次请求耗时：54毫秒 | lixinai01 | http-ni
```